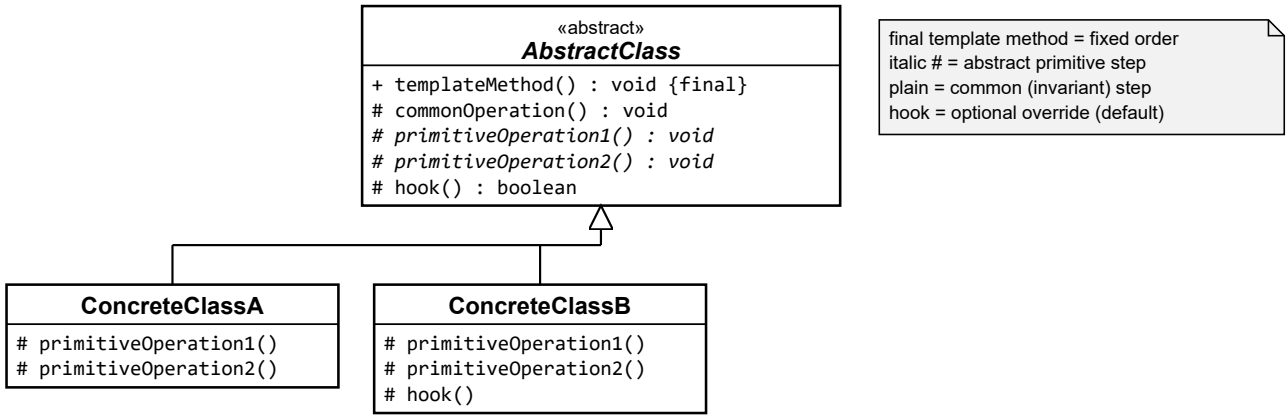
07 Template Method

Behavioral

Intent (GoF)	Define the **skeleton of an algorithm** in an operation (superclass), **deferring some steps to subclasses**. Subclasses redefine certain steps **without changing the algorithm's structure**.
Use when	Several classes follow the **same sequence of steps**; some steps are common, some differ per type; the **order must be controlled centrally**; you want to avoid duplicating the whole algorithm. **Same process + different steps = Template Method.**
Trigger words	"follow the same overall process / common workflow", "standard steps", "however X and Y are different for each type", "standardize the workflow while allowing custom behaviour", "every level must follow the same procedure", "enforce the procedure", "same salary components with deviations".
Participants	**AbstractClass**: `final` template method (fixed order), common (invariant) steps, `abstract` primitive (variant) steps, optional **hooks** · **ConcreteClass**: implements the variant steps only.
OOP used	Inheritance · abstract class/method · overriding · `final` (can't override the order) · access modifiers (`protected` steps, `private` common steps) · polymorphism ("Hollywood principle": parent calls the child).

UML — generic




```
public abstract class DataProcessor {            // lecture
example
    public final void processData() {             // TEMPLATE
METHOD
        loadData();      // common
        cleanData();     // varies
        analyzeData();   // varies
        generateReport();// common
    }
    private void loadData()      { System.out.println("Loading
data"); }
    protected abstract void cleanData();
    protected abstract void analyzeData();
    private void generateReport(){
System.out.println("Generating report"); }
}
public class CSVDataProcessor extends DataProcessor {
    protected void cleanData()   { System.out.println("Cleaning
CSV data"); }
    protected void analyzeData() { System.out.println("Analyzing
CSV data"); }
}
DataProcessor p = new CSVDataProcessor();
p.processData();  // Loading, Cleaning CSV, Analyzing CSV,
Generating report
```

Method kind	Modifier	Role
Template method	`public final`	fixes the order; subclasses can't change it
Common step	`private`/concrete	written once, shared
Primitive step	`protected abstract`	subclass **must** implement
Hook	`protected` with default body	subclass **may** override (e.g. `wantsEmail()` returns true)

Why is `processData()` final? To prevent subclasses from changing the sequence; they can customize `cleanData()` / `analyzeData()` but not rearrange the algorithm. **Which method controls the flow?** The template method. **Why only one abstract method in BasicEngineering (lecture)?** Only `SpecialPaper()` differs per course; the other steps are common, so subclasses override only that one.

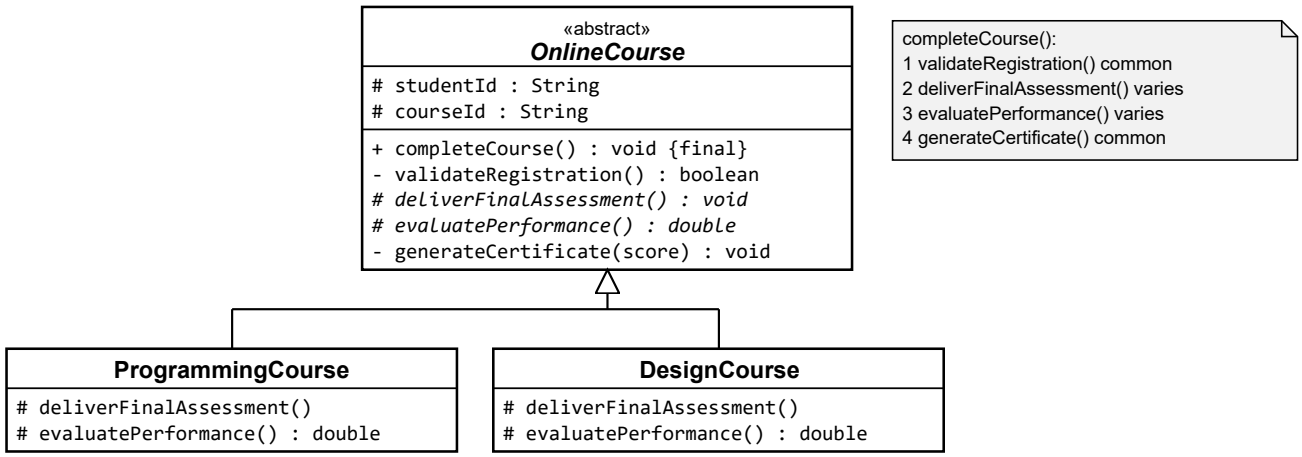
Advantages / challenges → mechanism

Advantages	Challenges (slides)	Mechanism to overcome
Controls the flow; clients can't change it	Client cannot direct the sequence of steps	If order must vary per request → use **Builder** (or Strategy) instead; use hooks for optional steps
Common code centralized ⇒ **no duplication**, reuse	A subclass may override (hide) a parent method	Make template `final`, common steps `private`/`final`; only primitives `protected abstract`
Consistency across subclasses; easy to add a new type (OCP)	More subclasses ⇒ scattered code, harder to maintain	Keep steps small; move shared variant code into an intermediate abstract class; document each step
Popular in frameworks (JUnit, Spring)	Rigid: changing the skeleton affects every subclass	Design the skeleton carefully; add new optional steps as hooks with default bodies

Questions & model answers

Online course completion: validate registration → deliver course-specific final assessment → evaluate performance → generate certificate. Programming = coding challenge + automated tests; Design = portfolio + instructor rubric. Pattern & why (2); UML (8); abstract class code (3); concrete classes (4). *Quiz 6*

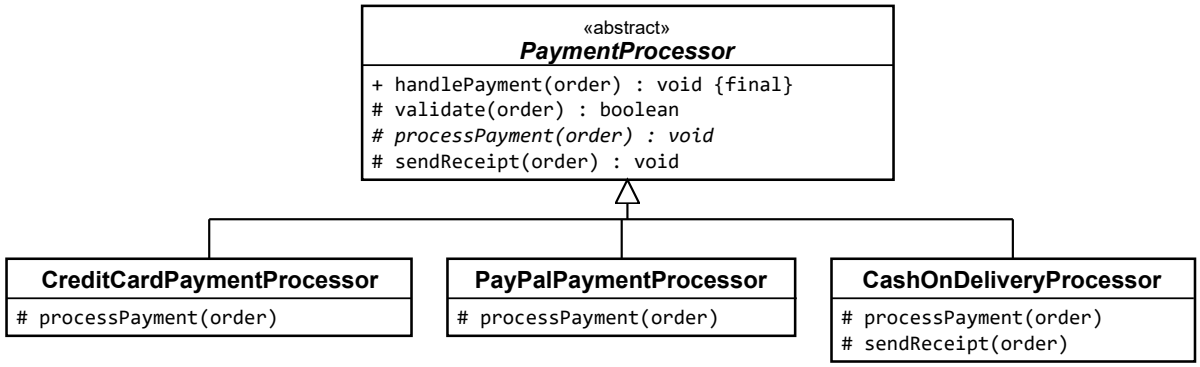
Template Method: every course follows the same completion sequence (must be enforced), but two steps vary per course type. The abstract class fixes the order in a `final completeCourse()`, implements the common steps once, and leaves `deliverFinalAssessment()` and `evaluatePerformance()` abstract.




```
public abstract class OnlineCourse {
    protected String studentId, courseId;
    public OnlineCourse(String s, String c) { studentId = s; courseId = c; }

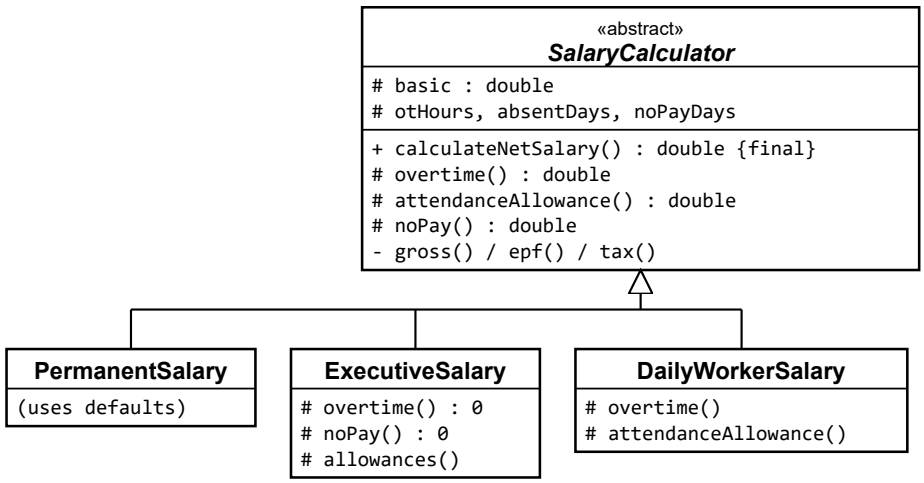
    public final void completeCourse() {                // TEMPLATE METHOD
        if (!validateRegistration()) { System.out.println("Not registered"); return; }
        deliverFinalAssessment();
        double score = evaluatePerformance();
        generateCertificate(score);
    }
    private boolean validateRegistration() {            // common
        System.out.println("Validating registration of " + studentId); return true;
    }
    protected abstract void deliverFinalAssessment();     // varies
    protected abstract double evaluatePerformance();      // varies
    private void generateCertificate(double score) {      // common
        if (score >= 50) System.out.println("Certificate issued for " + courseId + " (" + score + ")");
        else System.out.println("Not passed");
    }
}
public class ProgrammingCourse extends OnlineCourse {
    public ProgrammingCourse(String s, String c) { super(s, c); }
    protected void deliverFinalAssessment() { System.out.println("Coding challenge assigned"); }
    protected double evaluatePerformance() {
        int passed = AutoTestRunner.run(studentId), total = 20;     // automated tests
        return passed * 100.0 / total;
    }
}
public class DesignCourse extends OnlineCourse {
    public DesignCourse(String s, String c) { super(s, c); }
    protected void deliverFinalAssessment() { System.out.println("Submit design portfolio"); }
    protected double evaluatePerformance() {
        return Instructor.gradeWithRubric(studentId);               // instructor rubric
    }
}
OnlineCourse c = new ProgrammingCourse("S001", "Java101");  c.completeCourse();
```

E-commerce payments (Credit Card, PayPal, COD): validate → process (varies) → send receipt. Define abstract `PaymentProcessor` with `validate()`, `processPayment()`, `sendReceipt()`, `handlePayment()`; implement `CreditCardPaymentProcessor`. *2025 Q4.1(a), 4.2–3.3 (7)*




```
public abstract class PaymentProcessor {
    public final void handlePayment(Order o) {                    // template method
        o.setId(OrderIdGenerator.getInstance().nextId());          // Singleton (Note 01)
        if (!validate(o)) { System.out.println("Invalid payment details"); return; }
        processPayment(o);
        sendReceipt(o);
    }
    protected boolean validate(Order o) { return o.getAmount() > 0 && o.getCustomer() != null; }
    protected abstract void processPayment(Order o);
    protected void sendReceipt(Order o) { System.out.println("Receipt sent for " + o.getId()); }
}
public class CreditCardPaymentProcessor extends PaymentProcessor {
    protected boolean validate(Order o) { return super.validate(o) && CardValidator.luhnCheck(o.getCardNo()); }
    protected void processPayment(Order o) {
        System.out.println("Charging card " + o.getMaskedCard() + " LKR " + o.getAmount());
        BankGateway.authorizeAndCapture(o.getCardNo(), o.getAmount());
    }
}
```

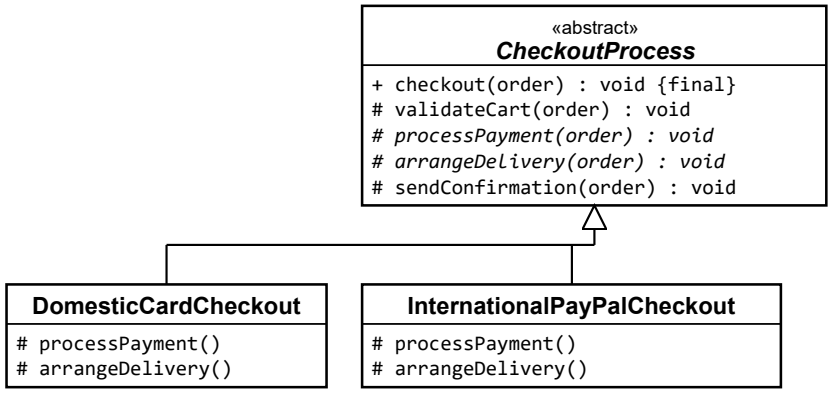
Supermarket pay: executives (basic + bonus + allowances), daily workers (8h/day, overtime, >200h attendance allowance), temporary staff (daily allowance); all pay 8% EPF. Pattern; class diagram; challenges; "code duplication can be achieved" — agree? *2022 Q2; 2023 Q1 (salary components)*



Template Method: one fixed calculation sequence (gross → EPF 8% → deductions → net) in `final calculateNetSalary()`; EPF, tax, gross and net formulas are common; components that differ (overtime, allowances, no-pay) are primitive/hook methods. Executive overrides `overtime()` and `noPay()` to return 0 (2023 deviation). **Challenges:** rigid order; many subclasses; parameter (rate) changes. **Mechanisms:** hooks with defaults; keep rates (EPF 8%, 12%, ETF 3%, tax threshold 100,000) in a **Singleton configuration** loaded from DB/properties so they change without code changes; intermediate abstract classes for shared variants. **Code duplication:** the statement is wrong as worded — Template Method **avoids** duplication: common formulas exist once in the superclass, subclasses contain only differences. Some small duplication may remain if two subclasses override a step the same way → move it to the parent or a shared intermediate class.

```
public final double calculateNetSalary() {
    double gross = basic + attendanceAllowance() + overtime();         // OT = basic*hrs/(30*8)
    double epfEmp = gross * Config.get().epfEmployee();                 // 0.08
    double tax = Math.max(0, (gross - 100000) * 0.06);
    return gross - (epfEmp + noPay() + tax);                            // noPay = basic*days/30
}
```

E-commerce checkout: validate cart → process payment (methods vary) → arrange delivery (domestic/international) → send confirmation. Best pattern; how abstract classes are used. *2024 Q4.2, 4.4 (10)*



Template Method. `CheckoutProcess` is an **abstract class** because it must hold *implemented* common steps (validateCart, sendConfirmation) ***and*** declare abstract steps — an interface couldn't enforce the order or share code. It can't be instantiated; the `final checkout()` fixes the sequence. Concrete subclasses supply payment and delivery. Payment method itself could be a separate Strategy/Factory object passed in to avoid a subclass for every payment × delivery combination. Exceptions (4.3) → Chain of Responsibility (Note 08). Authorization levels (4.5) → Proxy/CoR.

Every support level must log request → attempt to solve → log resolution, each with its own solving logic. Enforce the procedure. *Quiz 7 Q4 (4)*

Combine **Template Method with the CoR handler**: the abstract `SupportHandler` gets `public final void handle(req)` = `logRequest()` → `boolean solved = attemptToSolve(req)` (abstract) → `logResolution()` → if not solved, pass to `next`. L1/L2/L3/Specialist only implement `attemptToSolve()`, so no level can skip logging. (Code in Note 08.)

University exams (theory, programming, practical): register → prepare → conduct → evaluate → publish. *Lecture case 1* · **Order types (standard/premium/international) differ only in shipment preparation.** *Lecture case 2*

Both Template Method: `ExaminationProcess.conductExamination()` final; register & publish common; prepare/conduct/evaluate abstract. `OrderProcessor.processOrder()` final; validate, price, payment, notify common; `prepareShipment()` abstract.

vs Strategy: Template = inheritance, varies *steps* inside a fixed algorithm (compile time). Strategy = composition, swaps the *whole algorithm* at runtime. **vs Factory Method:** a factory method is often one step of a template method. **Mistakes:** template method not `final` · making every step abstract (no reuse) · using an interface for the base · client calling steps individually · drawing dashed (implements) instead of solid (extends) arrows.